سوال دوم) پردازش سیگنال های ECG:

LSTM:

بخش اول:

در این مرحله، ابتدا داده‌های مورد نیاز از مخزن Kaggle با اســـتفاده از کتابخانه‌ی opendatasets دریافت می‌شوند. این داده‌ها شامل سیگنال‌های الکتروکاردیوگرام (ECG) مربوط به بیماران مختلف هستند.

پس از دریافت داده‌ها، ســیگنال‌های مربوط به لید MLIIاز هر فایل CSV مربوط به بیماران اســتخراج می‌شــود. دلیل انتخاب لید MLII این اســت که در پردازش ســیگنال‌های ECG به‌عنوان یکی از لیدهای اصــلی اســتفاده می‌شود که کیفیت مناسبی برای تحلیل دارد.

در تابع load_mlII_signals:

- ابتدا تمام فایل‌های CSV مربوط به بیماران خوانده می‌شوند.
- از بین ستون‌های موجود در هر فایل، فقط ستون MLIIانتخاب می‌شود.
- برای یکنواخت‌ســازی طول ســیگنال‌ها، اگر طول ســیگنال کمتر از ۶۵۰,۰۰۰ نمونه باشــد، با صــفر پد می‌شود و اگر بیشتر باشد، برش داده می‌شود.
- در نهایت سیگنال‌های استخراج‌شده به صورت آرایه‌ای دوبعدی برمی‌گردند که هر ستون نشان‌دهنده‌ی سیگنال یک بیمار است.

بخش دوم (Data Augmentation):

با توجه به اینکه شبکه‌های عصبی عمیق برای یادگیری مؤثر نیاز به داده‌های زیاد و متنوع دارند، در این بخش از پروژه تکنیک‌های مختلفی برای افزایش تنوع سیگنال‌های ECG استفاده شده است.

- **تکنیک های مورد استفاده:**

۱. افزودن نویز تصادفی (Jittering):

با افزودن نویز گوسی با میانگین صفر و انحراف معیار $\sigma = 0.05$، سیگنال‌ها کمی متفاوت می‌شوند تا مدل به نویزهای طبیعی در داده مقاوم شود.

۲. تغییر مقیاس (Scaling):

دامنه سیگنال‌ها به‌صورت تصادفی در بازه‌ی (۰.۹ تا ۱.۱) ضرب می‌شود.

۳. تحریف زمانی (Time Warping):

طول سیگنال به‌طور موقت افزایش می‌یابد و سپس مجدداً به طول اولیه برش داده می‌شود. این روش باعث تغییر ظریف در شکل کلی سیگنال می‌شود.

هدف از این افزایش تنوع، کمک به مدل برای یادگیری ویژگی‌های عمومی‌تر از سیگنال‌های ECG و جلوگیری از overfitting است.

در تابع expand_dataset، این تغییرات برای هر سیگنال اصلی اعمال شده و به ازای هر سیگنال دو نسخه‌ی افزایش‌یافته تولید شده است (قابل تنظیم با num_augments_per_signal).

پس از تعریف توابع افزایش داده، مجموعه سیگنال‌های اولیه‌ی ECG که در ماتریس mlII_matrix ذخیره شده بودند، با استفاده از تابع expand_datasetگسترش داده می‌شوند. در اینجا به ازای هر سیگنال اصلی، فقط یک نسخه‌ی افزوده‌شده (Augmented) ایجاد می‌شود. در نتیجه اگر در ابتدا N سیگنال وجود داشته، اکنون تعداد سیگنال‌ها به 2N افزایش پیدا کرده‌اند (نسخه‌ی اصلی + نسخه‌ی افزوده).

```
Original shape: (650000, 46)
Augmented shape: (650000, 92)
```

بخش سوم (آماده‌سازی داده‌های ورودی و جلوگیری از Data Leakage):

در این مرحله، هدف تبدیل سیگنال‌های بلند به قطعات کوتاه‌تر با نویز مصنوعی برای آموزش شبکه عصبی است. همچنین، داده‌ها به‌صورت اصولی به سه دسته‌ی آموزش(Train) ، اعتبارسنجی (Validation) و آزمون (Test) تقسیم می‌شوند تا از نشت داده (Data Leakage) جلوگیری شود.

- **مراحل:**

۱. **تقسیم سیگنال به قطعات کوتاه و ایجاد نویز مصنوعی:**

در تابع create_clean_noisy_sequences، کل سیگنال به بازه‌هایی با طول ثابت (مثلاً ۵۰ نمونه) تقسیم می‌شود.

برای هر قطعه:

- یک نسخه‌ی تمیز (clean) ذخیره می‌شود.
- یک نسخه‌ی نویزی (noisy) با اضافه کردن نویز گوسی با $\sigma = 0.05$ تولید می‌شود.
- این دو نسخه به ترتیب به‌عنوان **ورودی مدل (X)** و **هدف مدل (y)** در نظر گرفته می‌شوند.

۲. **تقسیم داده‌ها به سه مجموعه مستقل:**

برای جلوگیری از **نشت داده(Data Leakage)** ، تقسیم‌بندی داده‌ها براساس شناسه‌ی بیماران انجام می‌شود، نه به‌صورت تصادفی از بین قطعات سیگنال. فایل patient_split.jsonشامل لیستی از شماره بیماران برای هر کدام از سه دسته‌ی train, val, testاست. با استفاده از این تقسیم‌بندی، داده‌های هر بیمار فقط در یکی از سه مجموعه قرار می‌گیرند.

۳. **تبدیل هر مجموعه به نمونه‌های کوتاه‌شده و نویزی:**

در نهایت، شکل نهایی داده‌ها چاپ می‌شود تا از صحت فرآیند اطمینان حاصل شود:

```
Train: (780000, 50, 1)
Val:   (104000, 50, 1)
Test:  (104000, 50, 1)
```

بخش چهارم (آماده‌سازی داده‌های ورودی و جلوگیری از طراحی مدل Autoencoder مبتنی بر LSTM):

در این بخش، یک معماری Autoencoder طراحی شده که وظیفه‌ی آن بازسازی سیگنال‌های ECG تمیز از نسخه‌های نویزی آن‌هاست. ساختار مدل شامل مراحل زیر است:

- **ورودی:** مدل داده‌هایی با ابعاد)طول قطعه، ۱ ویژگی (را دریافت می‌کند.
- **رمزگذار(Encoder)** : از یک لایه LSTM استفاده شده تا ویژگی‌های مهم هر قطعه استخراج و فشرده‌سازی شوند.
- **بخش میانی(Bottleneck)** : بردار خروجی رمزگذار توسط RepeatVectorتکرار می‌شود تا به طول اولیه بازگردد.
- **دیکدر(Decoder)** : لایه‌ی LSTM دوم توالی را از بردار فشرده بازسازی می‌کند و در نهایت TimeDistributed(Dense(1))خروجی نهایی را برای هر گام زمانی تولید می‌کند.

این مدل برای یادگیری بازسازی سیگنال‌های تمیز از داده‌های نویزی بسیار مناسب است و با هدف پروژه که حذف نویز از سیگنال‌های ECG است، کاملاً مطابقت دارد.

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 50, 1)	0
lstm_8 (LSTM)	(None, 64)	16,896
repeat_vector_4 (RepeatVector)	(None, 50, 64)	0
lstm_9 (LSTM)	(None, 50, 64)	33,024
time_distributed_4 (TimeDistributed)	(None, 50, 1)	65

Total params: 49,985 (195.25 KB)

Trainable params: 49,985 (195.25 KB)

Non-trainable params: 0 (0.00 B)

بخش پنجم (آموزش مدل و انتخاب بهترین اندازه Batch):

در این بخش، مدل Autoencoder طراحی‌شده با استفاده از مجموعه داده‌ی آموزش و اعتبارسنجی آموزش داده می‌شود. در این فرایند، چند مقدار مختلف برای اندازه‌ی دسته (Batch Size) تست شده تا بهترین عملکرد انتخاب شود.

- **مراحل:**

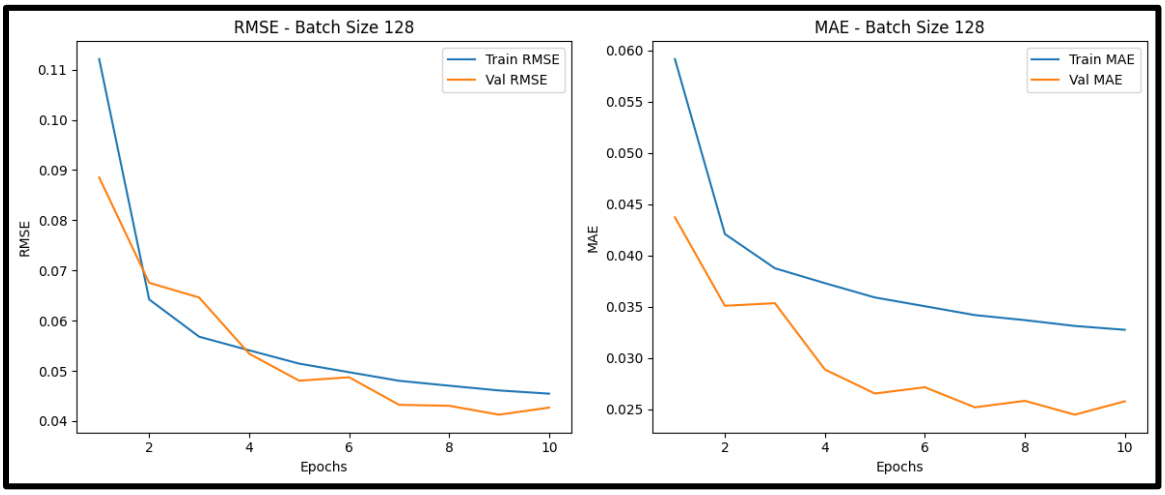
۱. **تنظیم پارامترها:** سه مقدار متفاوت برای اندازه‌ی دسته انتخاب شده‌اند: ۳۲، ۶۴ و ۱۲۸. هدف، بررسی تأثیر این پارامتر بر دقت مدل است.
۲. **کامپایل مدل:** مدل با تابع هزینه‌ی MSEو معیارهای ارزیابی RMSEو MAEکامپایل شده است. این انتخاب‌ها برای مسائل بازسازی عددی مناسب هستند.
۳. **استفاده از Callback ها:**
   - ModelCheckpoint: ذخیره بهترین مدل در طول آموزش براساس مقدار کمینه‌ی val_loss
   - EarlyStopping: جلوگیری از overfitting با توقف زودهنگام پس از ۳ دوره بدون بهبود درval_loss

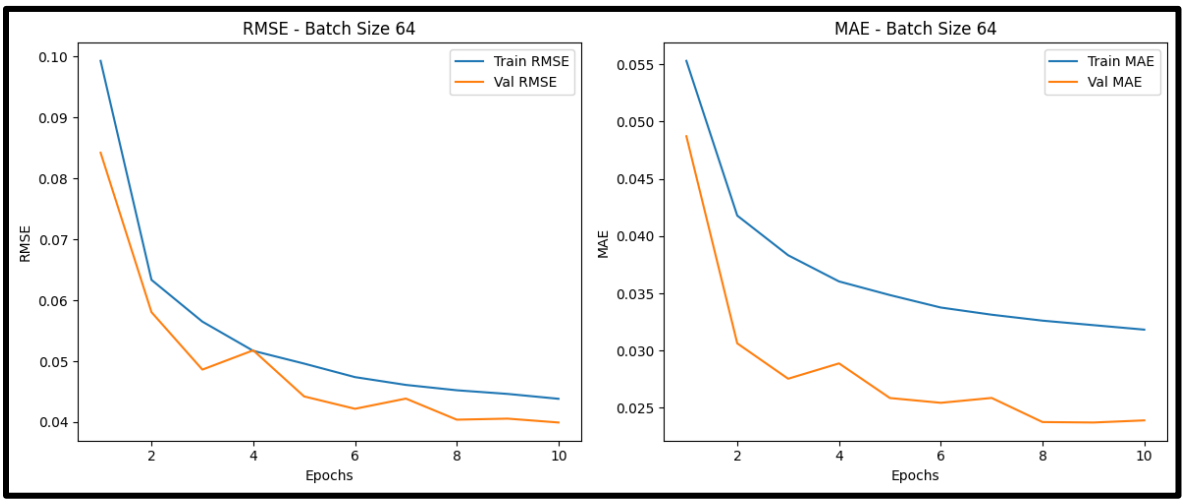
٤. **آموزش مدل:** برای هر مقدارbatch size ، مدل به مدت حداکثر ۱۰ دوره (epoch) آموزش داده شده و تاریخچه آموزش ذخیره شده است.
٥. **ارزیابی مدل:**
   - بعد از آموزش، بهترین مدل ذخیره‌شده برای هر batch size لود شده.
   - عملکرد آن بر روی مجموعه آزمون (Test Set) بررسی شده با محاسبه‌ی RMSE بین سیگنال‌های بازسازی‌شده و تمیز.
٦. **انتخاب بهترین مدل:** مدلی که کمترین مقدار val_lossرا داشته به‌عنوان بهترین مدل ذخیره و نتایج خلاصه‌سازی شده‌اند.

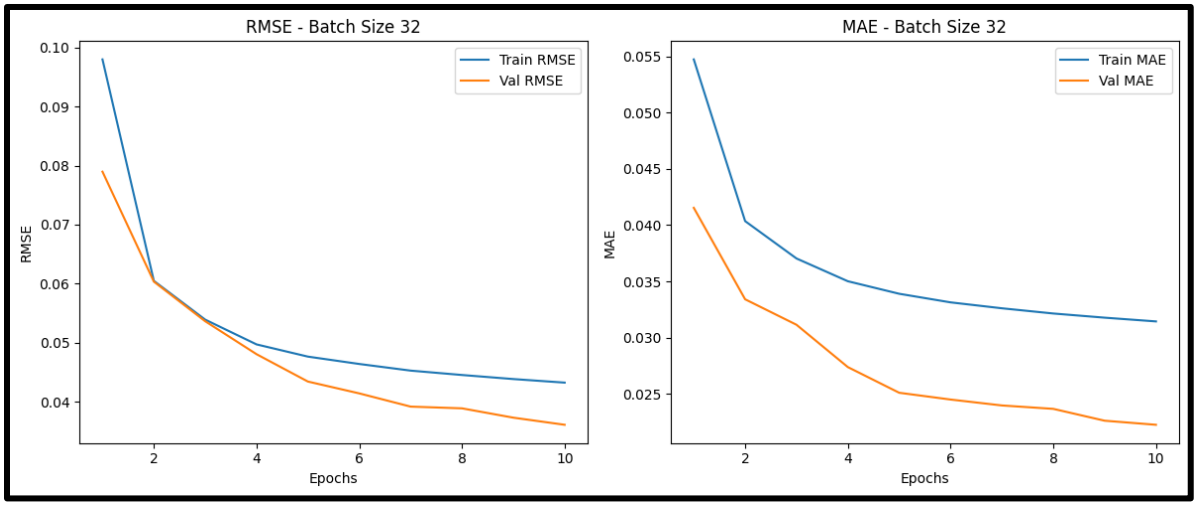
```
 Training with batch size 32...
Epoch 1/10
24373/24375 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - MAE: 0.0716 - RMSE: 0.1339 - loss: 0.0199
Epoch 1: val_loss improved from inf to 0.00624, saving model to /content/best_lstm_model_bs32.keras
24375/24375 ━━━━━━━━━━━━━━━━━━━━ 272s 11ms/step - MAE: 0.0716 - RMSE: 0.1339 - loss: 0.0199 - val_MAE: 0.0415 - val_RMSE: 0.0790 - val_loss: 0.0062
Epoch 2/10
24373/24375 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - MAE: 0.0415 - RMSE: 0.0627 - loss: 0.0039
Epoch 2: val_loss improved from 0.00624 to 0.00364, saving model to /content/best_lstm_model_bs32.keras
24375/24375 ━━━━━━━━━━━━━━━━━━━━ 330s 11ms/step - MAE: 0.0415 - RMSE: 0.0627 - loss: 0.0039 - val_MAE: 0.0334 - val_RMSE: 0.0604 - val_loss: 0.0036
Epoch 3/10
24373/24375 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - MAE: 0.0376 - RMSE: 0.0549 - loss: 0.0030
Epoch 3: val_loss improved from 0.00364 to 0.00288, saving model to /content/best_lstm_model_bs32.keras
24375/24375 ━━━━━━━━━━━━━━━━━━━━ 316s 11ms/step - MAE: 0.0376 - RMSE: 0.0549 - loss: 0.0030 - val_MAE: 0.0311 - val_RMSE: 0.0536 - val_loss: 0.0029
Epoch 4/10
24372/24375 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - MAE: 0.0352 - RMSE: 0.0500 - loss: 0.0025
Epoch 4: val_loss improved from 0.00288 to 0.00231, saving model to /content/best_lstm_model_bs32.keras
24375/24375 ━━━━━━━━━━━━━━━━━━━━ 277s 11ms/step - MAE: 0.0352 - RMSE: 0.0500 - loss: 0.0025 - val_MAE: 0.0274 - val_RMSE: 0.0481 - val_loss: 0.0023
Epoch 5/10
24370/24375 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - MAE: 0.0340 - RMSE: 0.0478 - loss: 0.0023
Epoch 5: val_loss improved from 0.00231 to 0.00189, saving model to /content/best_lstm_model_bs32.keras
24375/24375 ━━━━━━━━━━━━━━━━━━━━ 317s 11ms/step - MAE: 0.0340 - RMSE: 0.0478 - loss: 0.0023 - val_MAE: 0.0251 - val_RMSE: 0.0434 - val_loss: 0.0019
Epoch 6/10
24374/24375 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - MAE: 0.0333 - RMSE: 0.0469 - loss: 0.0022
Epoch 6: val_loss improved from 0.00189 to 0.00172, saving model to /content/best_lstm_model_bs32.keras
...
Batch  64 → Val Loss: 0.001595, Test RMSE: 0.032940
Batch 128 → Val Loss: 0.001704, Test RMSE: 0.035478

 Best Batch Size: 32 with Val Loss: 0.001305
```

نمودار مقدار Accuracy و Loss:







بخش ششم (ارزیابی نهایی مدل Autoencoder روی داده‌های تست):

در این مرحله، بهترین مدل ذخیره‌شده روی داده‌های تست ارزیابی می‌شود تا عملکرد نهایی آن در بازسازی سیگنال‌های تمیز از سیگنال‌های نویزی بررسی شود.

کار های انجام شده در این بخش:

۱. **بارگذاری مدل نهایی:** مدل LSTM انتخاب‌شده از مرحله‌ی قبل از مسیر ذخیره‌سازی فراخوانی می‌شود تا بر روی داده‌های تست اعمال شود.

۲. **پیش‌بینی خروجی:** مدل روی X_test اعمال شده و سیگنال‌های بازسازی‌شده توسط Autoencoder تولید می‌شوند.

۳. **محاسبه دستی معیارها:**

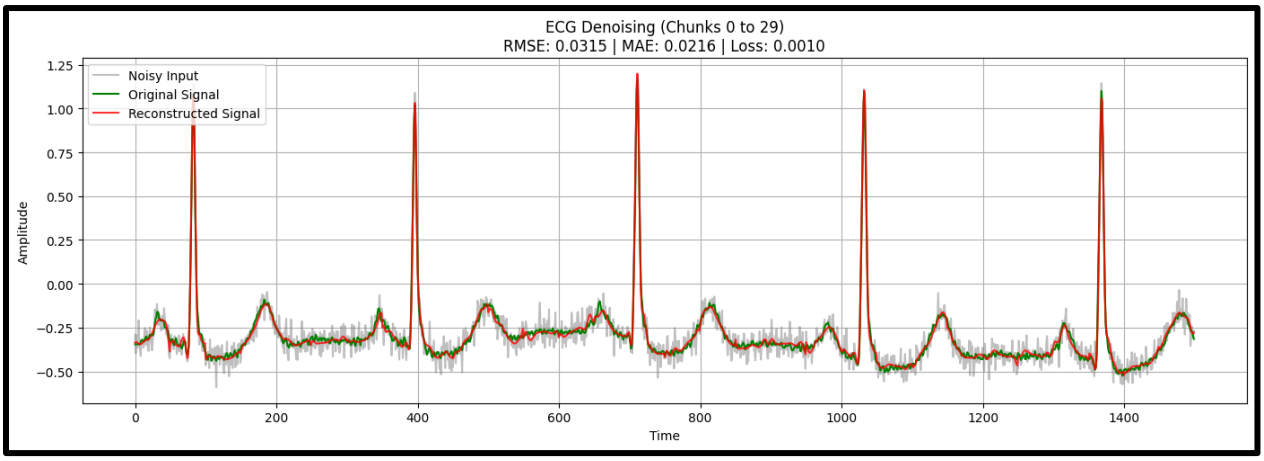
- RMSE (ریشه میانگین مربعات خطا): برای بررسی دقت کلی بازسازی
- MAE (میانگین قدر مطلق خطا): برای بررسی اختلاف میانگین بین مقادیر بازسازی‌شده و واقعی

۴. **استفاده از تابع evaluate:** علاوه بر محاسبات دستی، از تابع evaluate مدل برای به‌دست آوردن مقادیر loss و متریک‌های تعریف‌شده (RMSE و MAE) استفاده شده تا صحت ارزیابی تأیید شود.

```
 Loaded model from: /content/best_overall_lstm_model.keras
3250/3250 ━━━━━━━━━━━━━━━━━━━━ 14s 4ms/step

 Evaluated by model.evaluate():
Test Loss : 0.000992
Test RMSE : 0.031504
Test MAE  : 0.021565
```

در نهایت عملکرد مدل بازسازی سیگنال ECG را رسم کردیم:



GRU:

بخش اول:

در این بخش، سیگنال‌های ECG مربوط به لید MLII از فایل‌های CSV موجود در دیتاست MIT-BIH Arrhythmia بارگذاری شده‌اند.

- اگر سیگنال کمتر از ۶۵۰,۰۰۰ نمونه داشته باشد، با صفر padding شده و اگر بیشتر باشد، بریده می‌شود تا همگی طول یکسان داشته باشند.
- سپس سیگنال‌ها به صورت ماتریس NumPyبا شکل (650000, N)ذخیره شده‌اند که N تعداد بیماران یا رکوردهاست.

این داده‌های پردازش‌شده مستقیماً برای آموزش مدل GRU استفاده می‌شن، بنابراین این مرحله پایه‌ی کل فرآیند محسوب می‌شه.

فرایند Data Augmentation را مانند LSTM برای GRU نیز تکرار میکنیم. این مرحله به مدل GRU کمک می‌کند تا در مواجهه با تنوع سیگنال‌های واقعی، تعمیم‌پذیری بالاتری داشته باشد.

```
Original shape: (650000, 46)
Augmented shape: (650000, 92)
```

بخش دوم (شناسایی و اصلاح مکان دقیق قله‌های R در سیگنال ECG):

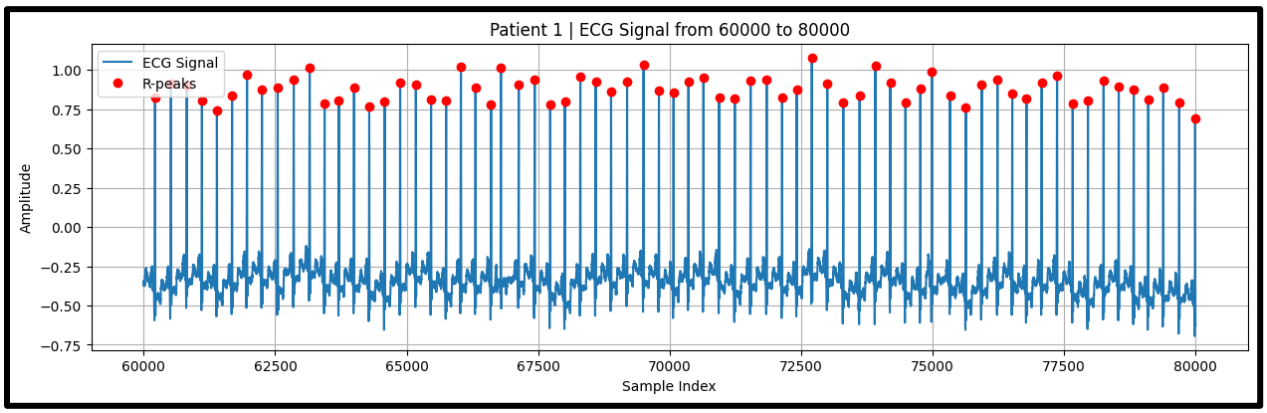
در این مرحله، قله‌های R که نمایان‌گر انقباض بطن‌ها در سیگنال ECG هستند، شناسایی شده‌اند. این قله‌ها اهمیت زیادی در تحلیل ضربان قلب دارند.

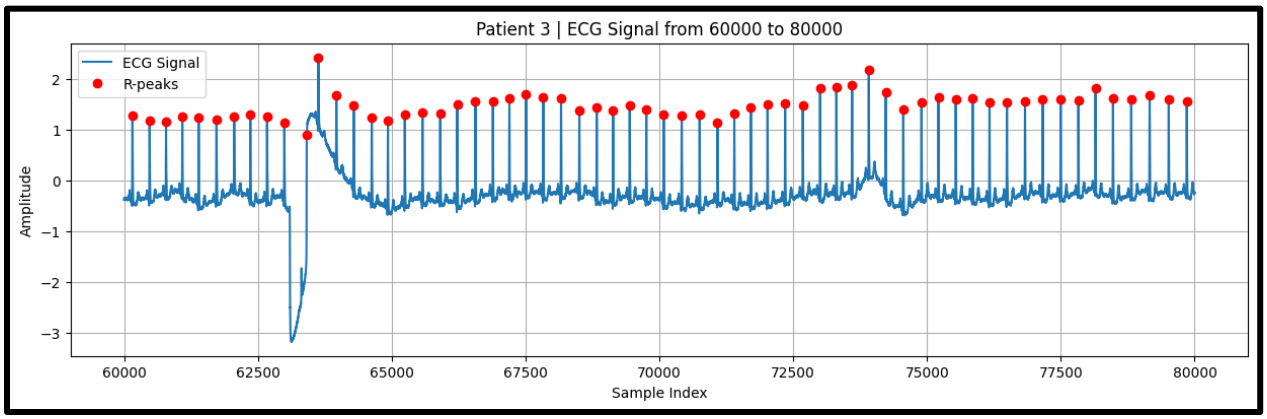
روش اجرا:

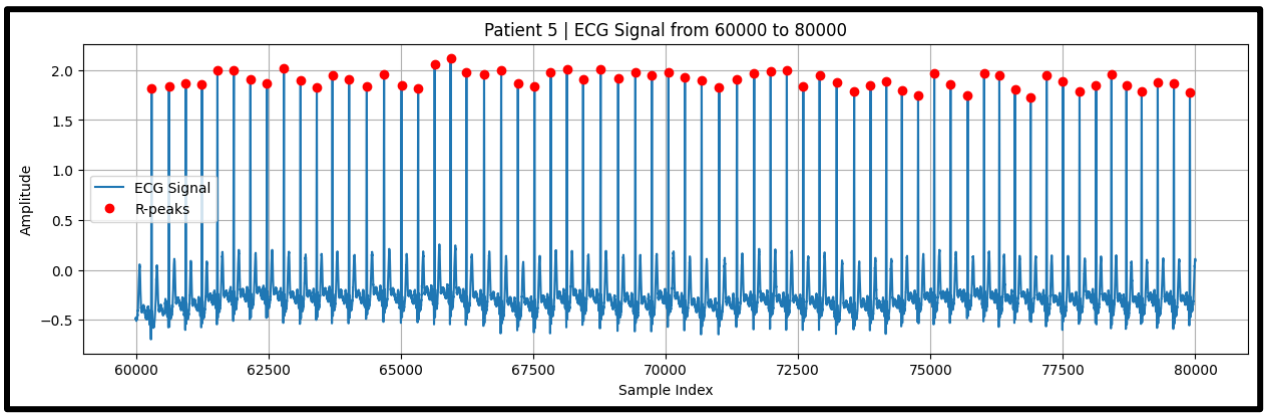
- ابتدا سیگنال‌ها نرمال‌سازی شدند (میانگین صفر و واریانس یک).
- سپس با استفاده از کتابخانه NeuroKit2 مکان اولیه‌ی R-peaks شناسایی شد.

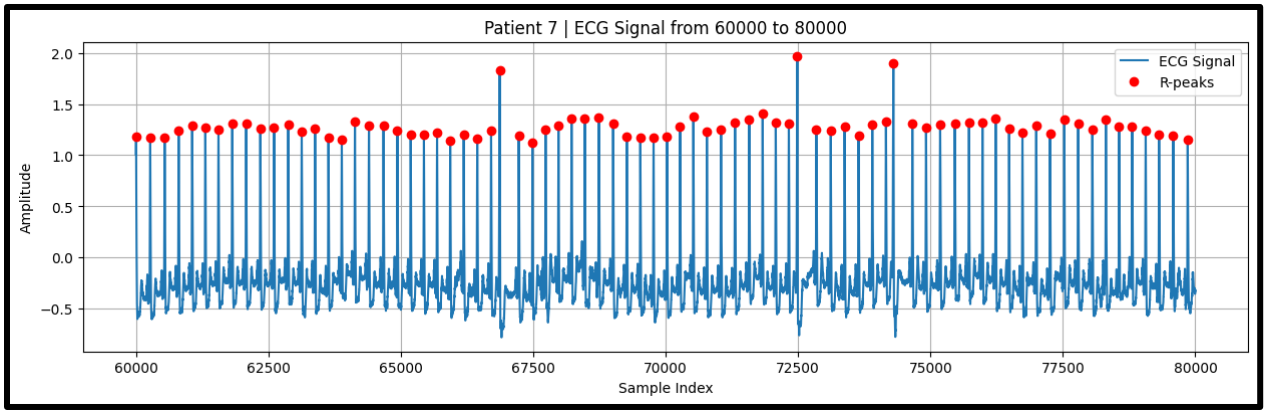
- برای دقت بالاتر، با استفاده از تابع refine_r_peaks، هر قله در یک پنجره‌ی اطراف محل اولیه بازبینی شده و دقیق‌ترین نقطه‌ی قله پیدا شده است.

این اطلاعات می‌توانند در مراحل بعدی (مثل ارزیابی فیزیولوژیک یا تحلیل نرخ ضربان قلب) مورد استفاده قرار گیرند، حتی اگر مستقیماً وارد آموزش مدل GRU نشده باشند.









بخش سوم (آماده‌سازی داده برای آموزش مدل GRU):

در این بخش، سیگنال‌های ECG و لیست R-peak های اصلاح‌شده برای تولید داده‌های ورودی و برچسب‌دار مورد استفاده قرار گرفتند. هدف، آموزش مدلی برای شناسایی ناحیه‌هایی است که حداقل یک R-peak در آن‌ها رخ داده است.

- مراحل:

۱. **تقسیم سیگنال‌ها به پنجره‌های ثابت**: هر سیگنال به قطعاتی با طول ثابت (مثلاً ۵۰ نمونه) تقسیم شد.

۲. **تولید برچسب برای هر پنجره**:

   - اگر حداقل یک R-peak در آن پنجره باشد، برچسب 1به آن داده شد.
   - در غیر این صورت، برچسب 0.

۳. **ساخت آرایه‌های X و y** :

   - X: شامل تمام پنجره‌های سیگنال با شکل (تعداد پنجره‌ها, طول پنجره, ۱)
   - Y: شامل برچسب باینری برای هر پنجره

سپس، داده‌ها بر اساس فایل تقسیم بیماران (patient_split.json) به سه دسته‌ی آموزش، اعتبارسنجی و تست تفکیک شدند تا از نشت داده بین بیماران جلوگیری شود

```
Train: (780000, 50, 1) (780000,)
Val:   (104000, 50, 1) (104000,)
Test:  (104000, 50, 1) (104000,)
```

بخش چهارم (معماری GRU و آموزش شبکه):

در این مرحله، یک مدل ساده‌ی GRU (Gated Recurrent Unit) برای حل مسئله‌ی باینری کلاس‌بندی طراحی شد. هدف مدل، شناسایی پنجره‌هایی از سیگنال است که حاوی حداقل یک R-peak هستند.

ساختار مدل به شرح زیر است:

۱. Input Layer
   - شکل ورودی) :طول پنجره, ۱ (که نشان‌دهنده سیگنال ECG در هر پنجره است.
۲. GRU Layer
   - شامل ۶۴ واحد GRU جهت استخراج ویژگی‌های زمانی از سیگنال.
۳. Dense Output Layer
   - یک نورون با تابع فعال‌سازی sigmoid برای تولید خروجی باینری (۰ یا ۱).

این مدل به صورت end-to-end طراحی شده تا از طریق یادگیری ویژگی‌های زمانی سیگنال ECG، موقعیت‌های احتمالی R-peak را شناسایی کند. و در ادامه، مدل با استفاده از داده‌های آماده‌شده آموزش داده خواهد شد.

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 50, 1)	0
gru (GRU)	(None, 64)	12,864
dense (Dense)	(None, 1)	65

Total params: 12,929 (50.50 KB)

Trainable params: 12,929 (50.50 KB)

Non-trainable params: 0 (0.00 B)

مدل GRU با سه مقدار مختلف batch_size شامل ۳۲، ۶۴ و ۱۲۸ آموزش داده شد تا تأثیر آن بر عملکرد مدل بررسی شود.

- **مراحل انجام‌شده:**

۱. **کامپایل مدل با:**
   - تابع خطاbinary_crossentropy
   - بهینه‌سازAdam
   - معیارهاAccuracy, RootMeanSquaredError, MeanAbsoluteError

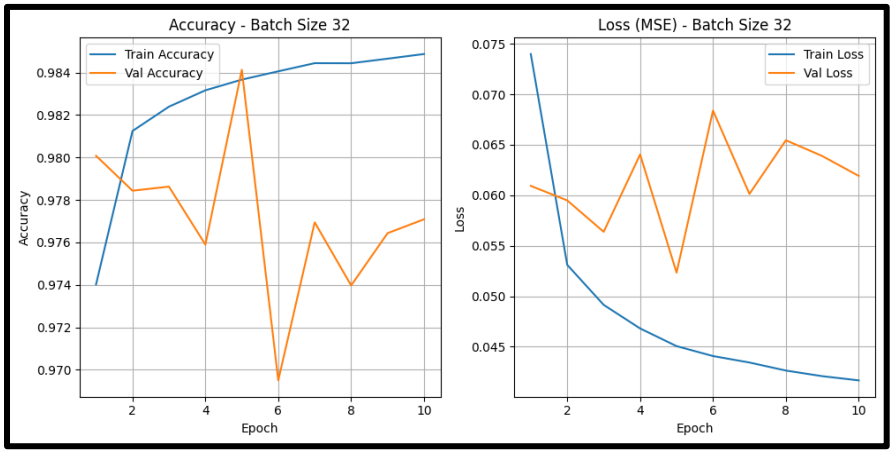
۲. **استفاده از callback ها:**
   - ModelCheckpoint: ذخیره بهترین مدل بر اساس کمترین مقدار val_loss
   - EarlyStopping: توقف زودهنگام در صورت عدم بهبود برای ۵ دوره متوالی

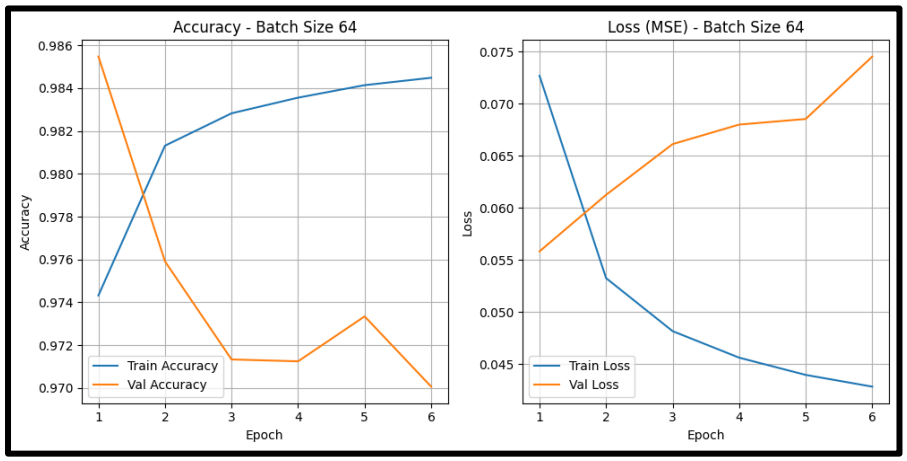
۳. **آموزش مدل‌ها** و ثبت history برای تحلیل بعدی.

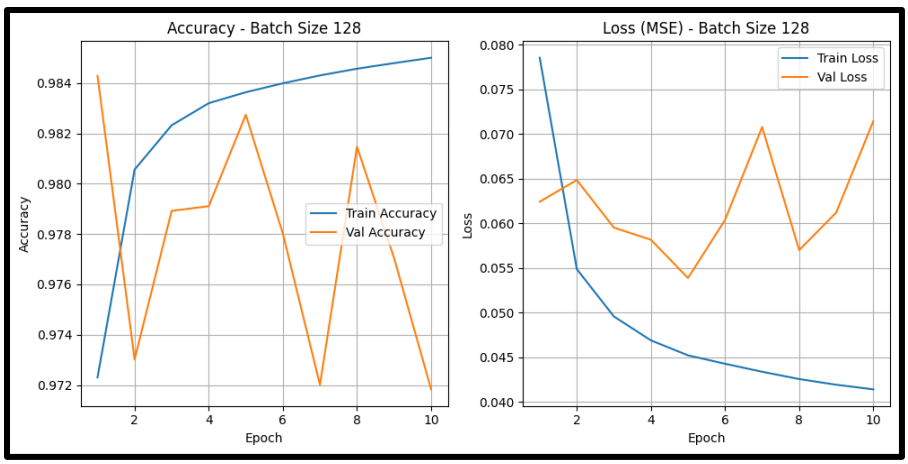
۴. **ارزیابی روی داده‌های تست** با محاسبه RMSE و انتخاب بهترین مدل.

بخش پنجم (نتایج بدست آمده):

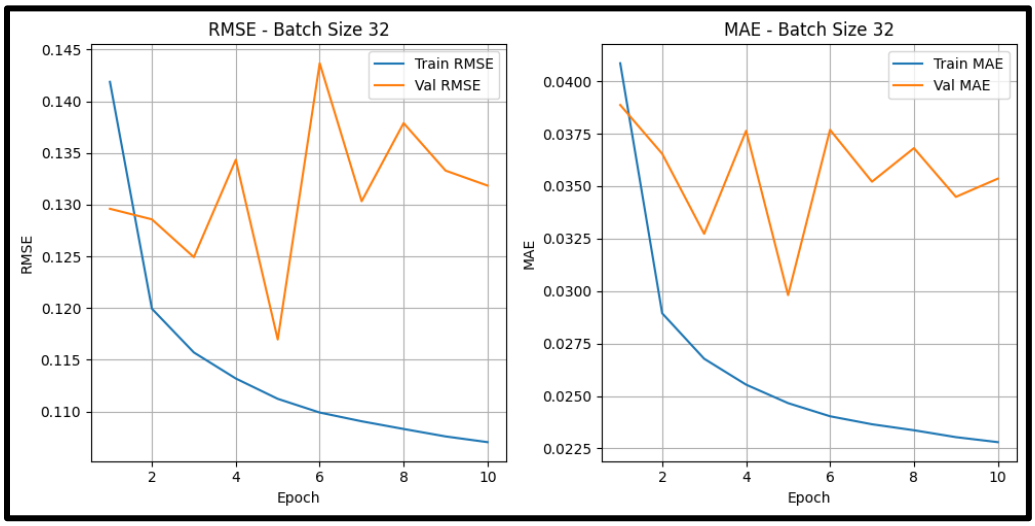
نتایج بدست آمده (Accuracy , Loss MSE):

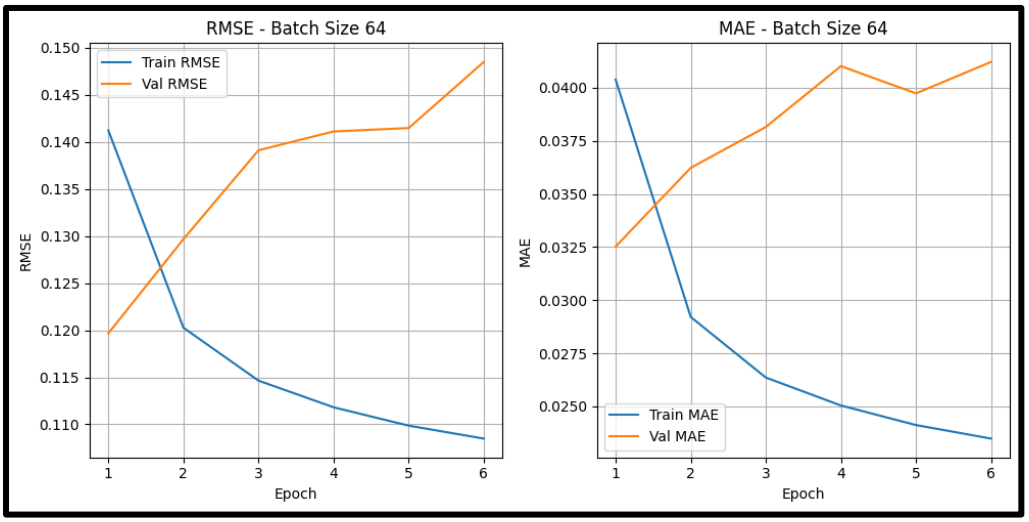


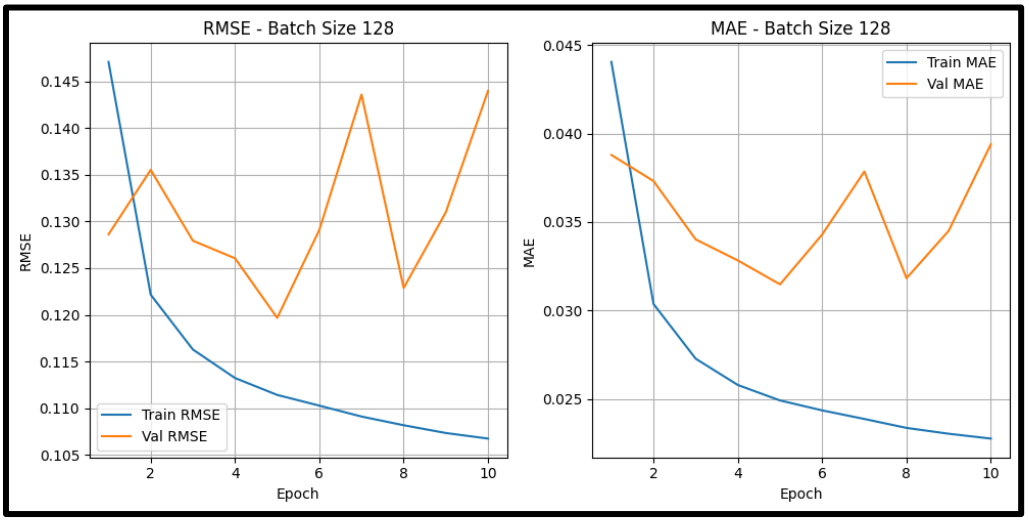




نتایج بدست آمده (RMSE , MAE):







و در پایان، عملکرد نهایی مدل GRU که بهترین مقدار batch_sizeرا داشت، با استفاده از معیارهای استاندارد در یادگیری ماشین بررسی شد.

معیارهای بررسی‌شده:

۱. **F1 Score**:
   - معیاری ترکیبی از دقت (precision) و یادآوری(recall)
   - مخصوصاً برای داده‌های نامتوازن مفید است.
   - مقدار نهایی به‌صورت عددی چاپ شد.

۲. **Confusion Matrix**:
   - ماتریسی شامل تعداد پیش‌بینی‌های صحیح و غلط در دسته‌بندی باینری:
     - True Positives (TP)
     - False Positives (FP)
     - True Negatives (TN)
     - False Negatives (FN)

۳. **Classification Report**:
   - گزارشی جامع شامل:
     - Precision
     - Recall
     - F1-Score
     - Support(تعداد نمونه‌ها در هر کلاس)

F1 Score: 0.8699

Classification Report:

	precision	recall	f1-score	support
0	0.9652	0.9922	0.9786	88056
1	0.9493	0.8027	0.8699	15944
accuracy			0.9632	104000
macro avg	0.9573	0.8975	0.9242	104000
weighted avg	0.9628	0.9632	0.9619	104000

